BEBOP-Gate: PRD, TRD, Architecture, Models and Datasets

BEBOP-Gate: Product Requirements Document (PRD)

Version 1.0 | 3 Nov 2026 | Status: draft for founder review | Consistent with BEBOP_Research_and_Decision.pdf

1. Summary

BEBOP-Gate is a safety check that sits between an AI's plan and the action. From a camera clip it measures whether a physical action fits (MVP: a block passing through a narrow gap) and returns **PASS, FAIL or UNDECIDED**. When it cannot tell, it names the missing measurement. It never approves an action only because an AI model suggested it.

Why now: NHTSA's March 2026 engineering analysis of Tesla FSD (EA26002) centres on a system failing to detect its own degraded visibility. Research preprints from 2026 (VAP-TAMP, AsgardBench) report that VLM-based planners handle occlusion and state tracking poorly. Broader recovery loops already exist (VAP-TAMP), so the product is scoped to the part that appears under-served: **abstaining with a measured margin**. This is **[UNPROVEN]** until our benchmark runs.

2. Goals and non-goals

Goals (MVP)	Non-goals
Correct PASS/FAIL on clear scenes	Autonomous driving or any safety-certified use
Return UNDECIDED, not a guess, when evidence is weak	Robot control or motion execution
Show the evidence behind every verdict	3D reconstruction, general physics simulation
Mark a saved assumption invalid when its inputs change	Training or fine-tuning a new model
Beat three baselines on false approvals (target, not a result)	Multi-agent orchestration, a production platform

3. Users and use cases

Primary user: a robotics or computer-vision developer who needs to know why a camera-driven system said "go". **Secondary:** a hackathon judge or mentor who needs to see, in two minutes, that it works and how it fails.

ID	User story
US-1	As a developer I upload a clip and get a verdict plus the gap measurement and its error.
US-2	As a developer I see which assumptions the verdict depends on and which became invalid after a change.
US-3	When the verdict is UNDECIDED I am told which measurement would settle it.
US-4	As a judge I can run the benchmark and see a results table that compares BEBOP with three baselines.

4. Functional requirements

ID	Requirement	Priority
FR-1	Accept a 10-20 s video (MP4) or a set of frames, plus the object width and required margin.	Must
FR-2	Segment the moving object, obstacles and passage edges through a pluggable backend: SAM 3.1 via API, or local Grounding DINO + SAM 2.1, or OpenCV color segmentation if no Meta credits arrive.	Must
FR-3	Convert pixels to millimetres using a printed scale marker; report the gap with an uncertainty.	Must
FR-4	Return PASS, FAIL or UNDECIDED using the rule in the TRD.	Must
FR-5	Keep a ledger of assumptions with the evidence frames and masks that support each.	Must
FR-6	Re-score the ledger on each new frame; mark assumptions and dependent decisions invalid when inputs change.	Must
FR-7	For UNDECIDED, output the missing measurement (what, tolerance, view).	Must
FR-8	Run the benchmark and three baselines and print the results table.	Must
FR-9	Check one alternate route with the same verifier after a FAIL.	Should
FR-10	Plain-language explanation from Muse Spark (never used for the verdict).	Could

5. Non-functional requirements

- **Safety posture:** fail closed. Any missing input (no marker, no mask, API error) yields UNDECIDED, never PASS.
- **Reproducibility:** one command re-runs the benchmark; fixed seeds; stored raw SAM responses.
- **Latency:** under 30 s per scene on a laptop plus API (target, not measured).
- **Cost:** SAM 3.1 costs $2.50 per 1,000 images, $0.20 per 1,000 frames (50-frame minimum per video request); the 40-scene benchmark should cost a few dollars at most (estimate).
- **Privacy:** use only non-sensitive footage if using Contributor mode, which lets Meta train on your content.

6. Success metrics

Metric	Why it matters	Target (hope, not result)
False-approval rate (PASS when truth is FAIL), overall and on unanswerable scenes	A wrong "yes" is the dangerous error	Lower than all three baselines
Coverage (share of scenes answered, not UNDECIDED)	Stops trivial "always UNDECIDED" solutions	At least 70%
Stale-plan retention after a change	Tests the invalidation logic	Lower than baselines
Missed changes	Detection quality	Reported, no target
UNDECIDED resolved by one extra measurement	Shows the missing-measurement advice is useful	Reported, no target

7. Scope and timeline

The event deadline is **Nov 4, 9:00 PM ET** (Nov 5, 6:30 AM IST), so the build is about 24 hours. A ten-day timeline applies only if the schedule changes; the same scope would then gain more scenes and polish, not more features.

8. Risks and open questions

Risk	Mitigation
No API access, acceptance or credits (current situation)	Start on the OpenCV backend today; add local Grounding DINO + SAM 2.1; label any substitute models honestly (see Models doc, section 6)
Similar public repos (BONISA, SceneProof, LandProof)	Lead with the head-to-head false-approval benchmark, not the 'code decides' idea
SAM output (special-token text) is hard to parse	Smoke test first; keep raw responses; fallback segmentation
Perspective error breaks mm conversion	Fixed near top-down camera; marker on the same plane; state the limit
Too few scenes to be convincing	Cut to 24 scenes; report counts honestly
A VAP-style baseline matches BEBOP	Report it; describe the contribution as engineering

Open questions: Is your account accepted? What camera and rig will you use? Which object and gap sizes will you physically build?

9. Acceptance criteria

1. On the 5 clearest scenes the verdict matches ground truth.
2. Every verdict shows measurement, uncertainty, evidence frames and assumption status.
3. A forced failure (marker covered, API disabled) returns UNDECIDED.
4. The results table exists with real measured numbers, or the document says it was not completed.

BEBOP-Gate: Technical Requirements Document (TRD)

Version 1.0 | 3 Nov 2026 | Implements the PRD

1. Stack

Component	Choice	Notes
Segmentation	Meta SAM 3.1 via Meta Model API (`sam-3.1`, Responses API)	Text prompt in; boxes and masks out as special-token text that must be parsed. Requires `MODEL_API_KEY`. **Untested by us.**
Segmentation, no credits	Grounding DINO (`IDEA-Research/grounding-dino-tiny`) + SAM 2.1 (`facebook/sam2.1-hiera-small`) via Transformers, both Apache-2.0	Runs locally or on a free Colab/Kaggle GPU; not tested by us
Segmentation fallback	OpenCV color/threshold segmentation on colored blocks	Zero dependencies; keeps the demo alive
Local VLM (Baseline 1, explanations)	Ollama: `qwen3-vl:8b`, `gemma3:4b` or `llama3.2-vision`	Ollama cannot run SAM; label as substitutes for Muse Spark
Optional local SAM	`facebook/sam3` / `facebook/sam3.1` via Transformers (`Sam3Model`, `Sam3TrackerVideoModel`)	Gated weights, SAM License; access requests have been rejected for some users, so do not depend on it
Language model	Muse Spark 1.3 (`muse-spark-1.3`), optional	Explanations only; local alternative Muse-Glimmer-30B
Core	Python 3.11, NumPy, OpenCV (incl. `cv2.aruco`), Shapely	Measurement, geometry, verdict, ledger
UI	Streamlit	Replaces FastAPI + React: too heavy for 24 h
Tests	pytest	Unit tests plus a benchmark runner

API base URL `https://api.meta.ai/v1`, bearer auth, OpenAI-SDK compatible. Back off on HTTP 429 (start 500 ms, double, add jitter, honor `Retry-After`).

Backends: environment variables `SEGMENT_BACKEND` (`meta_api`, `hf_local`, `opencv`) and `VLM_BACKEND` (`meta_api`, `ollama`, `none`) choose the implementation behind `segment()` and `explain()`. All backends return the same `{masks, boxes, scores}` shape, and the verdict code is identical.

2. Data model

```
{
  "scene_id": "s017",
  "frames": ["f000", "f012", "f024"],
  "object_width_mm": 120, "margin_mm": 5,
  "measurement": {"gap_mm": 142.0, "sigma_mm": 6.0, "n_samples": 15},
  "assumptions": [{
    "id": "A1", "text": "gap >= object_width + margin",
    "status": "valid|invalid|unknown",
    "evidence": [{"frame": "f012", "mask_ref": "m_gap_f012.png", "prompt": "wooden block"}],
    "inputs_fingerprint": {"blocker": {"centroid": [412, 230], "mask_hash": "..."}}
  }],
  "decisions": [{"id": "D1", "route": "R1", "depends_on": ["A1"], "verdict": "PASS|FAIL|UNDECIDED"}],
  "missing_measurement": null
}
```

3. Algorithms

3.1 Pixels to millimetres. Detect an ArUco marker of known size with OpenCV; scale = marker_mm / marker_px. If the marker is not found, the scene is UNDECIDED.

3.2 Masks. For each of 3 frames and 5 prompt variants per object ("wooden block", "block", "small box" ...) call SAM; keep the highest-confidence match. If there are zero matches or conflicting multiple matches, record an instability flag.

3.3 Gap and width. Convert masks to polygons (`cv2.findContours`). Gap = Shapely distance between the two obstacle polygons. Object width = short side of the minimum-area rectangle of the object mask. Each sample gives a value; the mean is the estimate and sigma combines the sample standard deviation with a pixel-quantisation term (about 0.5 px) and the marker-scale error.

3.4 Verdict. Let required = width + margin and d = gap - required, with sigma_d = sqrt(sigma_gap^2 + sigma_width^2).

Condition	Verdict
Any stability gate fails (no marker, no mask, mean mask IoU across prompt variants below tau, API error)	UNDECIDED
d > k * sigma_d	PASS

d < -k * sigma_d	FAIL
Otherwise	UNDECIDED

Start with k = 2 and tau = 0.85, then tune **only on the calibration split** (see section 6).

3.5 Invalidation. Each assumption stores a fingerprint of its input masks. On a new frame, an assumption becomes invalid if any input's mask IoU against the reference falls below 0.8, its centroid moves more than a set pixel threshold, or the object disappears. Decisions that list the assumption in `depends_on` are flagged.

3.6 Missing measurement. If UNDECIDED because |d| < k * sigma_d, the required tolerance is sigma_target = |d| / k. Report: which quantity (gap edge or width), the tolerance needed, and the view that reduces error (unoccluded, top-down, closer). If |d| is smaller than the camera's measurement floor, report that a physical measurement is needed.

4. Interfaces

Interface	Contract
`segment(frame, prompt) -> {masks, boxes, scores}`	Wraps SAM 3.1 or the OpenCV fallback; stores raw responses
`measure(masks, marker) -> {gap_mm, sigma_mm, flags}`	Pure function
`verdict(measurement, rule) -> PASS/FAIL/UNDECIDED`	Pure function, no model calls
`ledger.update(frame) -> changes`	Pure function over stored fingerprints
`explain(ledger) -> text`	Optional Muse Spark call; output never read by `verdict`

5. Repository layout

```
bebop/ segment.py measure.py verdict.py ledger.py baselines.py explain.py
app/ streamlit_app.py
bench/ scenes/ truth.csv run_bench.py report.py
tests/ test_measure.py test_verdict.py test_ledger.py
```

6. Evaluation protocol

- ~40 scenes: 3 gap classes (clear pass, marginal, clear fail) x conditions (clean, hand-occluded, blur, low light, lookalike distractor, object moved mid-clip). At least 10 are unanswerable by design.
- Ground truth: calipers or ruler, 3 repeated measurements per gap.
- **Split:** about 12 scenes for calibration (k, tau, prompts), about 28 held out for the reported test. Never tune on the test set.
- Baselines: (1) Muse Spark yes/no; (2) SAM mask + geometry, single shot, binary; (3) VAP-style reimplementation (5 paraphrases, majority vote, sufficiency check, forced binary answer). Label (3) a reimplementation.
- Report all numbers from the stored run; no hand-edited results.

7. Testing

pytest unit tests for each pure function; a fixture of 5 synthetic images with known gaps (drawn in OpenCV) so geometry and verdict are tested without the API; fault-injection tests (no marker, empty mask, API timeout) must all return UNDECIDED.

8. Security and operations

API key in an environment variable, never committed. Cache raw API responses to avoid repeat cost. Avoid Contributor mode for private footage. Log every verdict with its inputs for audit.

9. Known limitations

Planar-scene assumption (camera near top-down, marker on the same plane); single object type; no motion planning; thresholds tuned on a small calibration set; SAM 3.1 output format and access not yet verified by us.

BEBOP-Gate: Architecture Document

Version 1.0 | 3 Nov 2026 | Read with the PRD and TRD

1. Principle

The model proposes, the measurement decides. Foundation models find things in images and write explanations. A small deterministic core turns measurements into a verdict. An AI model's suggestion is never evidence that an action is safe. Any missing or unstable input makes the system fail closed (UNDECIDED).

2. System context and layers

```
Phone/camera clip  +  object width, margin
       |
       v
+------------------ PERCEPTION (models) ------------------+
|  segment.py: Meta SAM 3.1 via Model API                 |
|  backends: meta_api | hf_local (DINO+SAM2.1) | opencv   |
+--------------------------+------------------------------+
                           | masks, boxes, scores, raw responses
                           v
+------------------ MEASUREMENT (deterministic) ----------+
|  ArUco scale -> mm | polygons (Shapely) | gap, width    |
|  uncertainty from 3 frames x 5 prompt variants          |
+--------------------------+------------------------------+
                           | gap +/- sigma, stability flags
                           v
+------------------ DECISION CORE (deterministic) --------+
|  verdict.py  PASS / FAIL / UNDECIDED  (+ missing        |
|  measurement)         ledger.py  assumptions, evidence, |
|  invalidation, dependent decisions                      |
+-------------+------------------------+-----------------+
              |                        |
              v                        v
+-- EXPLANATION (optional) --+   +-- UI + BENCHMARK ---------+
|  Muse Spark: plain-language |   |  Streamlit: overlay,      |
|  text; never read by verdict|   |  verdict, ledger, message |
+-----------------------------+   |  bench/: 3 baselines +    |
                                  |  BEBOP, results table     |
                                  +---------------------------+
```

3. Components

Component	Responsibility	Trust level
Perception	Find object, obstacles, passage edges	Untrusted: output is checked by stability gates
Measurement	Convert masks to millimetres with an error estimate	Trusted, pure functions, unit-tested
Decision core	Verdict, ledger, invalidation, missing measurement	Trusted, pure functions
Explanation	Human-readable text	Untrusted, display only
UI	Show evidence	Display only
Benchmark	Run baselines and BEBOP on the same scenes	Produces all reported numbers

4. Runtime flow

1. Load clip and parameters; sample 3 frames.
2. Detect the scale marker; if missing, stop with UNDECIDED.
3. For each frame and prompt variant, call SAM; store raw responses.
4. Measure gap and width; compute sigma and stability flags.
5. Run `verdict`; write assumption A1 and decision D1 to the ledger with evidence.
6. On each later frame, re-score the ledger; invalidate A1 and flag D1 if inputs changed.
7. If UNDECIDED, compute and display the missing measurement; if FAIL, check one alternate route.
8. Optionally call Muse Spark for an explanation.

5. Failure modes and handling

Failure	Effect	Handling
API down, 429, malformed response	No masks	Backoff, then OpenCV fallback or UNDECIDED
Marker hidden	No scale	UNDECIDED
Masks disagree across prompts	Unstable perception	UNDECIDED, ask for a clearer view
Gap within error of threshold	Cannot resolve	UNDECIDED with required tolerance
Camera tilted	Biased mm	Documented limit; reject scenes beyond tilt limit

6. Architecture decisions

ADR	Decision	Reason	Trade-off
1	Verdict is deterministic code	Auditable, testable, avoids model-driven false approvals	Less flexible than a VLM judge
2	Hosted SAM 3.1 first, local weights not required	HF weights are gated and some requests were rejected	Depends on API access
3	Streamlit instead of FastAPI + React	24-hour window	Less polished UI
4	No model training	No time or labelled data; zero-shot SAM is enough to test the claim	Fixed model limits
5	Ledger is an explanation aid, not a novelty claim	VAP-TAMP re-verifies before each action	Smaller claimed contribution
6	Pluggable perception backend; OpenCV first if no Meta credits	Credits and gated weights may not arrive in time; the verifier must not depend on them	Weaker Meta-model story; substitutes must be labelled honestly

7. Deployment

Single laptop running Python and Streamlit, calling `https://api.meta.ai/v1`. Without Meta credits: local Grounding DINO + SAM 2.1 (Colab/Kaggle GPU if needed) and an Ollama vision model for explanations. Optional: Muse-Glimmer-30B on a 24 GB GPU. No servers, databases or queues; scene results are files on disk.

BEBOP-Gate: Best Hugging Face Models and Datasets (as of 3 Nov 2026)

Updated 4 Nov 2026 with a no-credits plan (section 6). Findings come from web searches in this session; I did not download or run anything. "Verified" means seen on a model card, paper or project page; "check" means confirm before relying on it.

1. Do we need to train anything?

No. BEBOP-Gate uses zero-shot segmentation plus a deterministic verifier. The only "fitting" is **calibrating two thresholds** (k and tau) on a small calibration split of our own scenes. Fine-tuning SAM would add risk and no evidence for the claim. Public datasets below are for **stress-testing and sanity checks**, not training.

2. Models

Role	Model	Why	License / access	Status
Primary segmentation	facebook/sam3.1 (also via Meta Model API as `sam-3.1`)	Open-vocabulary text prompts; SAM 3.1 adds faster multi-object video tracking. HF support via `Sam3Model` and `Sam3TrackerVideoModel` (docs)	SAM License; **gated**; some users report rejected access (example thread)	Verified exists; use the API as the main path
Segmentation fallback	SAM 2.1 (`facebook/sam2.1-hiera-*`)	Mature; MOSE val J&F 71.8 for the tiny checkpoint per its package page (source)	Check license on the card	Fallback only
Cheapest fallback	OpenCV color segmentation	No model, no access	n/a	Keeps demo running
Explanations (optional)	Muse Spark 1.3 via API	Same API and key	Hosted; Contributor mode trains on your data	Verified in docs
Local explanations (optional)	meta-models/Muse-Glimmer-30B	Apache-2.0, text+image input, about 17 GB quantised for a 24 GB GPU; released Aug 10, 2026	Apache-2.0 plus usage policy	From press coverage; benchmark claims are self-reported
Optional metric depth	depth-anything/DA3METRIC-LARGE	Metric depth from one image (0.35B); useful if the camera is tilted	Apache-2.0 per its card; other DA3 sizes may be CC BY-NC 4.0	Optional; we prefer the printed marker

Not recommended: Muse Spark Open (816B MoE, impractical to host in a day); Depth Pro and UniDepth V2 (extra license constraints, UniDepth is non-commercial); fine-tuning any model.

3. Datasets and footage

Purpose	Dataset	What it gives	Limits
Primary (build this)	**Our own BEBOP-Clearance scenes (~40)**	Ground-truth gaps, occlusions, unanswerable cases built to test abstention	You must capture it; this is the only dataset that tests our exact claim
Occlusion stress test	MOSEv2 (5,024 videos; occlusion, low light, weather); MOSE v1	Shows how segmentation degrades: SAM 2 falls from 76.4% to 50.9% on v2 per the paper	Video object masks, not gap measurements; MOSE v1 is CC BY-NC-SA 4.0, non-commercial; check v2 terms
Degraded-visibility check	ACDC (4,006 adverse images: fog, night, rain, snow; 5,509 annotated incl. normal)	Real adverse conditions and an uncertainty-aware segmentation task, matching the NHTSA theme	Driving scenes, not tabletop; check terms
Complex-scene video	SeCVOS	Videos with 30.2% object-disappearance rate	CC BY-NC-SA 4.0; optional
Plan-adaptation reference	AsgardBench (108 task instances, CC BY-SA 4.0 paper)	Optional side test of stale-state behaviour in a simulator	Household tasks, high-level actions
Synthetic corruptions	Generated by us with OpenCV (blur, low light, noise, occluding patches)	Free, reproducible, controlled difficulty	Not real-world

4. Footage capture guide (the dataset that matters)

- **Rig:** fixed phone on a tripod, near top-down, 1080p, 30 fps; printed ArUco marker of known size on the same plane.
- **Objects:** one block of known width; two obstacles forming a gap you can adjust.
- **Gap classes:** clear pass (about +30 mm), marginal (about +/-8 mm), clear fail (about -30 mm).
- **Conditions per class:** clean; hand occluding one edge; blur; low light; lookalike distractor; obstacle moved mid-clip.
- **Unanswerable scenes (at least 10):** deciding edge fully hidden, or gap within measurement error. Correct output: UNDECIDED.
- **Ground truth:** calipers or ruler, 3 measurements per gap, stored in `truth.csv` with the camera setup.

- **Split:** about 12 scenes for calibration, about 28 for held-out testing. Do not tune on the test set.
- **Clip length:** 10-20 s each; name files `sNNN_class_condition.mp4`.

5. Caveats

Model-card benchmark numbers (especially Muse Glimmer) are self-reported and I could not confirm independent replication. Access to gated SAM weights is not guaranteed. Dataset licenses above are what I saw in this session; confirm before redistributing anything.

6. Plan B: no Meta API credits (use this if your key or credits do not arrive)

Check your credit status at dev.meta.ai first. One participant's README says the $150 credit must be claimed by Nov 4 ET, which I could not verify from an official page. Everything below runs without that key.

Job	Local model (Hugging Face or Ollama)	Why	Verified?
Text-prompted masks	**Grounding DINO** (IDEA-Research/grounding-dino-tiny or `-base`) turns "wooden block" into boxes, then **SAM 2.1** (facebook/sam2.1-hiera-small) turns boxes into masks	Both Apache-2.0 and in Transformers (`Sam2Model`); SAM 2.1 is itself a Meta model. Small checkpoint is light (a third-party page reports 74 MiB weights, 53 ms warm on a GPU)	Licenses and Transformers support seen on model cards and repos; I did not run them
Zero-model fallback	OpenCV color/threshold segmentation of colored blocks	No download, no GPU, deterministic; the demo cannot fail on access	Standard OpenCV
Free GPU	Google Colab or Kaggle notebook for the two models above	CPU-only laptops are slow for Grounding DINO	Not tested by me
Baseline 1 and explanations (Ollama)	`ollama run qwen3-vl:8b` (about 8 GB VRAM), `gemma3:4b` (4 GB), `llama3.2-vision` (11B, Meta)	Image + text in; Ollama lists these vision models	From third-party guides, not tested; Ollama **cannot run SAM**, so use Transformers for masks
Meta-family local option	meta-models/Muse-Glimmer-30B, Apache-2.0, about 17 GB quantised	Keeps a Meta model in the story	Press coverage only; needs a 24 GB GPU

Config switch: set `SEGMENT_BACKEND=meta_api | hf_local | opencv` and `VLM_BACKEND=meta_api | ollama | none`. The verdict code does not change.

Honesty rule: if you use substitutes, say so. Label baselines "Qwen3-VL (Ollama)" instead of "Muse Spark", and do not claim SAM 3.1 or Muse Spark if you did not call them. The hackathon scores use of Meta's models, so a local SAM 2.1 / Llama story is weaker than the hosted one; the measured false-approval table matters more in that case.

Recommended order with no credits: (1) OpenCV segmentation to get the verifier and benchmark working today; (2) add Grounding DINO + SAM 2.1 as the "real" perception backend if time remains; (3) add an Ollama VLM only for Baseline 1 and explanations.

7. Competitor note (hackathon repos, 4 Nov 2026)

Public repos such as BONISA (refuses to give a measurement when checks fail), SceneProof and LandProof already pair Meta models with deterministic verification. I read only their README openings. Details are in BEBOP_Hackathon_Participants_OnePage.pdf.